

AlphaPi-One 项目 · 踩坑与经验总结

适用范围：ESP32 / MicroPython 板卡逆向 + 机器人遥控 + 树莓派部署。

目的：把本项目踩过的坑固化成清单，做下一个产品时直接照抄/避雷。

每条格式：现象 → 根因 → 解决。

关联：[board_map.md](#)（引脚/寄存器速查） ·
[../projects/AlphaPiCar/README.md](#)（完整可复用示例） ·
[README.md](#)（文档索引）

0. 十条最重要的（先看这个）

1. 遥控运行时别开串口监视器；调试脚本也要考虑"一连就重启"。
2. `sys.modules` 会缓存模块 → 改完代码（或 `sys.modules.pop("xxx")`）才生效。
3. 串口桥缓冲会溢出、设备会 `MemoryError` → 处理。
4. `=`，整体解码会在第一个 `=` 截断 → 必须。
5. `bytearray(40960)` 在导入重模块后分配会失败 → 复用 `boot.py` 里提前分配的缓冲。
6. `bytes` 没有 `.hex()`；`socket` 没有 `SO_BROADCAST`；`socket` 功能有限。
7. 设备上的 `.py` 往往自带注释/文档，比反汇编 `.mpy` 省事得多。
8. 能读写多个寄存器 + 有语义算法，就不是 PCF8574 那种简单 IO 扩展器。
9. 对未知 I2C 设备，先 `scan()` + `readfrom_mem()`；（可能直接驱动电机）。
10. (GBK 报错) → 用 `ascii + backslashreplace` 或写文件。

1. MicroPython / ESP32 板端（串口 / REPL / 内存）

1.1	脚本长期 <code>NO_RAW</code> / <code>ALL_FAILED</code> ，设备像"卡死"	用错 RTS/DTR 触发了错误复位时序，把设备卡在 boot	<code>raw REPL</code> ；一次连接内多次重试进
1.2	灌一大段代码后只收到 <code>OK</code> 或残缺输出	串口桥 TX 缓冲被"回显" + 输出"冲爆"，数据丢失	分块回发（32 字节/块）+ 边发边丢弃回显 + 用 <code>START/END</code> 标记框住真正输出
1.3	上传新代码后行为没变化	设备，模块被 <code>sys.modules</code> 缓存	复位板子；REPL 临时： <code>import sys; sys.modules.pop("模块名", None)</code>
1.4	<code>MemoryError: memory allocation failed</code>	堆碎片化；导入重模块后再申请大块内存	大缓冲尽早分配（见 1.5）；或分块处理； <code>gc.collect()</code>
1.5	<code>bytearray(40960)</code> 偶发失败	同上	<code>boot.py</code> <code>static_buf</code> (<code>getattr(__main__, "static_buf", None)</code>)，别再 new 一份
1.6	<code>'bytes' object has no attribute 'hex'</code>	MicroPython <code>bytes</code> 无 <code>.hex()</code>	用 <code>ubinascii.hexlify(b).decode()</code>
1.7	<code>AttributeError: 'module' object has no attribute 'SO_BROADCAST'</code>	MicroPython <code>socket</code> 精简	不要设 <code>SO_BROADCAST</code> ，直接 <code>sendto(广播地址)</code>
1.8	Windows 控制台打印中文报 <code>UnicodeEncodeError: 'gbk'</code>	控制台是 GBK	输出 <code>str(x).encode("ascii", "backslashreplace").decode()</code> ；或写文件再读
1.9	遥控中途打开串口监视器，车突然停了	开串口复位了板子	运行时 串口监视器
1.10	反汇编/上传脚本"时好时坏"	进 raw REPL 依赖时序，有随机性	所有交互脚本 循环重试，以标记/校验和确认成功

2. 通过串口拉取 / 上传文件 (base64 over REPL)

2.1	取大 <code>.mpy</code> 时设备报 <code>memory allocation failed</code>	一次性 <code>b2a_base64(整文件)</code> 爆堆	读取 + 每块单独 base64 打印
2.2	拼接后解码只得到第一块 (512 字节)	每块 base64 各自补了 <code>=</code> , 整体解码在第一个 <code>=</code> 处停止	逐行(逐块)解码再拼接字节, 不要整体解
2.3	上传大文件时编译报 <code>MemoryError</code>	一条超长字符串常量编译需要连续内存	: 多次 raw-REPL 执行, <code>f=open(...)</code> 在 <code>__main__</code> 全局里跨块保持
2.4	上传"成功"但内容不对	没校验	上传后打印 <code>len(open(f, 'rb').read())</code> 与本地比对
2.5	载荷里 <code>"\n"</code> 变成真换行导致语法错	宿主字符串转义	宿主里写 <code>"\\n"</code> (或严格用真换行, 不在设备字符串里放 <code>\n</code>)

3. `.mpy` 反编译

3.1	<code>mpy-tool.py</code> 报 <code>No module named 'makeqstrdata'</code>	缺依赖	同时下载 <code>py/makeqstrdata.py</code> 放到同目录
3.2	不确定 <code>.mpy</code> 版本	版本在文件头	头 4 字节 <code>4d 06 ...</code> : <code>byte0='M'</code> , <code>byte1=版本号</code> (本项目 =6)
3.3	想找函数签名/常量		<code>python mpy-tool.py -d x.mpy</code> , 看 <code>simple_name:</code> 与 <code>args:</code> ; 常量看 <code>LOAD_CONST_SMALL_INT</code>
3.4	反汇编很大看不过来		先 <code>search</code> 关键字 (<code>Pin</code> / <code>SPI</code> / <code>writeto</code> / 寄存器偏移) 再看上下文

4. 硬件逆向方法论（复用价值最高）

1.

:

os.listdir()

列出设备文件；文本

.py

直接打印（自带注释=最好的文档）。
2.

:

反汇编找

IMPORT

/

children

，理清模块依赖。
3.

:

用

machine.SoftI2C(任意GPIO)

扫 I2C；SPI/UART 从固件里读引脚。
4.

:
- 只响应 1 个"端口"、无寄存器 →

(如 PCF8574) 。
- 能按寄存器读写、有算法语义（位置/PID/校验） →

(本项目 0x20 = 电机控制器) 。
1.

:

readfrom_mem(addr, reg, n)

摸寄存器；
2.

:

固件里推断的寄存器/引脚，用实测（串口打印、发指令看反馈）确认。
3.

:

大段代码跑不动时，先跑最小片段，再逐段加回（本项目靠这招定位了 I2C

scan()

卡死）。

5. 本车硬件最终速查（AlphaPi-One）

主控	ESP32-S3, MicroPython, <code>.mpy</code> v6, 主频 240MHz
车体 UART	UART1 @ 160800, <code>TX=GPIO3</code> 、 <code>RX=GPIO0</code> （主状态协议 22 字节）
	SoftI2C <code>SCL=9, SDA=8</code> @100k; 从机
电机控制器寄存器	每电机一块： <code>+0x00</code> 速度(int32)、 <code>+0x04</code> 功率(int16)、 <code>+0x08</code> 位置(读)、 <code>+0x0C</code> 模式(int16)
电机基址	<code>L=0x20</code> 、 <code>R=0x10</code> 、 <code>Z=0x30</code>
I2C 写校验	速度写入带 4 字节取反校验；位置读回带取反校验
电机控制 API	模块 <code>autoMotionOne</code> ： <code>set_all_power</code> / <code>set_speed</code> / <code>set_raw_power</code> / <code>stop_motor</code> / <code>set_claw</code> / <code>get_position</code> / <code>go_distance</code> / <code>turn_round</code>
找不到 0x20 时	自动回退 UART2 @38400 (<code>tx=8, rx=9</code>)
	硬件 <code>SPI(2)</code> 20MHz： <code>SCK=41</code> <code>MOSI=42</code> <code>MISO=45</code> <code>CS=40</code> <code>DC=39</code> <code>RST=38</code>

WS2812 灯	<code>autoMotionOne</code> 内建 <code>NeoPixel(Pin(7), 4)</code> (板载灯效)
震动马达	<code>WritePwm(4, v)</code>
摇杆/按键	摇杆 <code>ADC 8(X)/7(Y)</code> 、电位器 <code>ADC 6</code> ；按键 <code>GPIO 10/11/12/13</code> (上拉低有效)
	<code>SetBrightness()</code> 只把亮度乘到 上；首次要能亮，必须 <code>setPixelColor()</code> ，否则黑×亮=黑
△ 参数类型坑	库统一用 <code>basic.DataStruct</code> 传参； <code>packRGBd()</code> 返回 <code>DataStruct</code> (别 <code>hex()</code> 它)

6. 网络与远程控制

6.1	板子热点在，但客户端拿不到 IP	/	板子 <code>openHotspot</code> 自带 DHCP；客户端等待并重连即可
6.2	板子"自发自收"UDP 收不到	<code>recv_udp</code> 前置条件： <code>ap_if.isconnected()</code> or <code>sta_if.isconnected()</code> ；无真实客户端连接时直接返回	必须有真实客户端连上热点/STA 已连接，端到端要真机测
6.3	板子自己给 <code>255.255.255.255</code> 发不回环	lwIP 自广播不回环	自测用 或 <code>127.0.0.1</code> ；对外用广播没问题
6.4	PC 双网卡互相干扰	两张网卡都想当默认网关	一张上网、一张连设备； <code>netsh wlan connect ... interface="WLAN 2"</code> ；配置文件要 <code>网卡</code> 添加
6.5	板子从 AP 改 STA 后热点消失	<code>connectWifi</code> 会 <code>ap_if.active(False)</code>	预期行为；STA 下板子进局域网，任何同网设备都能控
6.6	忘记板子 IP	DHCP 变动	① 用 <code>255.255.255.255:1000</code> (推荐) ② 路由器做 DHCP 保留 ③ 串口日志 <code>WIFI STA: (...)</code>

6.7	遥控长时间发指令但不动	丢包/顺序	方向指令 (如 10Hz) + 松手发"停"; 或板端做 (<code>AUTO_STOP_MS</code>)
6.8	上位机不知道小车是否在线	单向 UDP, 无反馈	板端定期广播心跳(hb) + 每条指令回执(ack), 上位机监听同一 UDP 端口, 按"最近收到时间"判定在线

(UDP :1000, JSON) :

```
{"token":"","message":"上/下/左/右/停/开爪/合爪/开灯/关灯/drive", "value":"..."}
```

- `drive` 的 value= "左轮,右轮" (各 -100~100) → 摇杆 。

7. 部署与运维

7.1	Windows 挂载盘看不到刚部署的文件	；且 <code>/home/pi</code> 是 <code>700</code>	资源管理器 / 重开窗口; <code>net use Y: /delete</code> 后重连; 确保用 挂载	
7.2	<code>netsh wlan</code> 提示 "no profile assigned to interface"	配置文件是 没 挂载的	<code>netsh wlan add profile filename=... interface="WLAN 2" user=all</code>	
7.3	自动化 SSH 到树莓派 (无 sshpass)	Windows 没 sshpass	用 <code>paramiko</code> : <code>pip install paramiko</code> ; sudo 用 `echo 密码 \	sudo -S cmd`
7.4	服务要开机自启/常驻		写 <code>systemd</code> 单元到 <code>/etc/systemd/system/</code> , <code>systemctl enable --now</code>	
7.5	树莓派跑脚本依赖第三方库		优先用 (本项目 <code>car_web.py</code> 零依赖, PC/Pi 通用)	

7.6	多项目混乱	无规范	约定: <code>/home/pi/Codes/<项目>/</code> + <code>README.md</code> + <code>systemd/</code> ; 顶层 <code>README.md</code> 做索引
7.7	Linux 上服务 <code>PermissionError</code> 无法绑定 1000 端口 (Windows 却正常)	, 非 root 用户不能绑	给 systemd 单元加 <code>AmbientCapabilities=CAP_NET_BIND_SERVICE</code> (最小授权); 或统一改用高端口
7.8	PowerShell 内联命令里的中文被破坏 (发"停"无效, 抓到的是乱码)	PowerShell 脚本按 ANSI 读取, 非 UTF-8	中文别走内联命令行: 写成 UTF-8 的 <code>.py/.ps1</code> 文件再执行, 或由 Python (<code>urllib</code>) 发送

8. 通用工程原则 (做下一个产品时)

- : 先把"能进 REPL"→"能读一个寄存器"→"能控一个动作"逐级打通, 再叠加。
- : 串口打印、`CMD` 回显、读回校验和——不要凭"应该可以"。
- : 上传后校验大小; 改行为后跑一次实测。
- : 一次只改一个地方; 二分法定位卡点。
- : 未知设备 ; 执行器 (电机/夹爪) 操作限速、限时、可急停。
- : UDP 不可靠场合 → 周期重发 + 死区(dead-man)自动停 + 参数带校验。
- : 项目自带 README (架构/协议/用法/排障) + 顶层索引; 踩坑单独成文。
- : 常驻能力用 systemd; 配置 (SSID/密码/端口) 集中到文件顶部易改。
- : 能标准库解决就不引第三方, 跨平台部署最省心。
- , 不堆在一个目录里; 本文件就是"跨项目经验库"。

9. 本项目沉淀的可复用工具 (`board_dump/`)

<code>scan_only.py</code>	raw-REPL 跑一段代码并抓输出	板子任意即席探查

<code>dump_src.py</code> / <code>decode_mpy.py</code>	拉取设备文件； <code>.mpy</code> 分块 base64 取回并解码	逆向任何 MicroPython 板
<code>repl_eval.py</code>	在板子 REPL 运行本地 <code>.py</code> 片段	快速验证想法
<code>upload_file.py</code> / <code>upload_chunked.py</code>	上传文件到板子（含分块版）	部署板端脚本
<code>mpy-tool.py</code> (+ <code>makeqstrdata.py</code>)	官方 <code>.mpy</code> 反汇编	看编译模块的签名/常量
<code>probe_car.py</code>	只读探测 I2C 外设寄存器	摸清任何 I2C 智能外设
<code>deploy_pi.py</code>	paramiko 部署到树莓派 + 装 systemd	树莓派批量部署
<code>car_web.py</code>	零依赖网页遥控（虚拟摇杆）	任何"HTTP→UDP/串口"控制面板

产品侧代码：

<code>remote_car.py</code>	板子 <code>/</code>	板端：连 WiFi(STA/AP) + 收 UDP + 驱动电机/爪子/灯
<code>car_web.py</code>	PC & 树莓派	网页遥控服务（含虚拟摇杆、急停）
<code>car_remote_client.py</code>	PC & 树莓派	命令行遥控
<code>/home/pi/Codes/AlphaPiCar/</code>	树莓派	项目目录（README + systemd 自启）

10. 做"下一个产品"的 Checklist（照抄即可）

- `[]`
- `[]` 复用 `boot.py` 的 `static_buf`，不自己 new 大内存
- `[]` 配置项（WiFi/端口/token/速度）集中到文件顶部
- `[]` 关键动作打日志（`CMD xxx`），便于无屏调试
- `[]` 加 `try-except`，防失控
- `[]` 模块更新后 `sys.modules`（或清 `sys.modules`）

- []
- [] 优先 发指令，别写死设备 IP
- [] 松手/断开自动发"停"
- [] 网页端做 大按钮 + 触摸友好
- []
- [] 代码放 `/home/pi/Codes/<项目>/`，带 `README.md`
- [] systemd 单元 + `enable --now`
- [] 顶层 `README.md` 更新索引
- [] 用 ， 减少依赖
- []
- [] 项目 README：架构图 / 协议表 / 用法 / 排障
- [] 把新踩的坑补进本文件（跨项目经验库）
- []
- [] 先 `os.listdir()` + 读 `.py` 源码
- [] 再 `.mpy` 反汇编；`SoftI2C` 扫总线；
- [] 设备定性：简单 IO 扩展器 vs 智能外设